Project Title: Warehouse PWA for Printing House (Inventory Management)

Generate a full-stack Progressive Web Application (PWA) for a printing house warehouse. The app manages finished goods (boxes of printed orders) and raw materials / tooling (paper, client-supplied stock, die-cut forms). It must work offline-first, be installable on mobile home screens, and be optimized for glove-friendly, large-button warehouse use.

---

1. BUSINESS CONTEXT & GOALS

Target users:

- Warehouse Operators – receive, store, search, and ship finished boxes and materials.
- Administrators – manage users, storage locations (shelves with QR), view statistics, audit logs, and system settings.

Pain points solved:

- Lost orders – search time reduced from 10 min to 10 sec.
- Wrong handwritten order numbers – OCR verification on sticker photo.
- Scattered orders (same order on different shelves) – each box has its own location.
- Lost client-supplied materials – always know where “Igor’s gray paper” is.
- No inventory transparency for management – real-time stats, long-staying orders, operator activity.

Core principles:

- Zero-manual-entry where possible – QR scanning, camera capture, OCR, visual search.
- Offline capability – all actions persist locally (IndexedDB) and sync when online.
- Two roles only – Operator (can do everything on the floor) and Admin (Operator + user/location/audit management).
- Admin can act as Operator – same UI, but all actions logged with Admin’s name.

---

2. TECHNOLOGY STACK (MANDATORY)

Area	Technology
Framework	React 18 + TypeScript + Vite

UI Library	Ant Design (adaptive, large controls)
Routing	React Router v6
Camera	Custom `useCamera` hook (getUserMedia)
QR Scanning	`html5-qrcode`
OCR	`tesseract.js` (recognize order number from sticker photo)
Offline Storage	Dexie.js (IndexedDB) – tables: orders, boxes, materials, locations, syncQueue
PWA	`vite-plugin-pwa` + `workbox` (manifest, service worker)
PDF Generation	`jspdf` + `jspdf-autotable` (QR labels, shipment acts)
QR Code in PDF	`qrcode`
Icons	`@ant-design/icons`
Styling	SCSS (global, mobile breakpoints)
Mock API	Axios + in-memory / localStorage mocks (with artificial delay)

Do NOT use any other UI libraries or heavy dependencies. Stick to the stack above.

---

3. PROJECT STRUCTURE (MUST FOLLOW)

Generate the following exact folder & file structure inside `src/`:

```text
src/
├── main.tsx
├── App.tsx
├── vite-env.d.ts
├── styles/
│   └── global.scss
├── context/
│   └── AuthContext.tsx
├── hooks/
│   ├── useCamera.ts
│   └── useOfflineSync.ts (stub)
├── components/
│   ├── QrScanner.tsx
│   ├── CameraCapture.tsx
│   ├── PhotoGallery.tsx
│   └── ProtectedRoute.tsx
├── layouts/
│   ├── OperatorLayout.tsx
│   └── AdminLayout.tsx
├── pages/
│   ├── Login.tsx
│   ├── operator/
│   │   ├── Dashboard.tsx
│   │   ├── BoxRegistration.tsx
│   │   ├── OrderSearch.tsx
│   │   ├── OrderDetail.tsx
│   │   └── MaterialsPage.tsx
│   └── admin/
│       ├── AdminStats.tsx
│       ├── AdminUsers.tsx
│       ├── AdminLocations.tsx
│       ├── AdminOrders.tsx
│       ├── AdminAudit.tsx
│       └── AdminSettings.tsx
```

```
├── services/
│   ├── api.ts
│   ├── auth.ts
│   ├── orders.ts
│   ├── boxes.ts
│   ├── materials.ts
│   ├── locations.ts
│   ├── users.ts
│   ├── audit.ts
│   └── settings.ts
├── store/
│   ├── db.ts                (Dexie schema)
│   └── sync.ts              (sync queue stub)
├── types/
│   ├── index.ts            (all shared interfaces)
│   └── api.ts
└── utils/
    ├── ocr.ts
    ├── pdf.ts
    └── qr.ts
```

Every file must be created with meaningful, typed, working code (mock data, no real backend). Use named exports consistently.

---

4. GLOBAL TYPES (DEFINE IN `src/types/index.ts`)

```typescript
export type Role = 'admin' | 'operator';
export type OrderStatus = 'active' | 'completed';
export type BoxStatus = 'in_stock' | 'shipped';
export type MaterialStatus = 'in_stock' | 'issued';
export type LocationType = 'permanent' | 'temporary';
export type CounterpartyType = 'client' | 'supplier' | 'both';

export interface User {
```

```
  id: string;
  name: string;
  email: string;
  role: Role;
  isActive: boolean;
  createdAt: string;
}

export interface Order {
  id: string;
  number: string;          // unique, user-friendly
  customer?: string;
  createdAt: string;
  status: OrderStatus;
  completedAt?: string;
}

export interface Box {
  id: string;
  orderId: string;
  numberInOrder: string;   // e.g. "3/8"
  quantity: number;
  locationType: LocationType;
  locationId?: string;     // UUID of permanent shelf
  tempLocationPhoto?: string; // base64
  tempLocationDesc?: string;
  status: BoxStatus;
  shippedAt?: string;
  shippedBy?: string;      // userId
  productPhotos: string[]; // base64
  stickerPhoto?: string;   // base64
  problem?: {
    type: string;
    description: string;
    photo?: string;
```

```
    createdAt: string;
  };
  createdAt: string;
  createdBy: string;
  synced?: boolean;        // for offline
}

export interface Material {
  id: string;
  type: 'raw' | 'client_supplied' | 'tool';
  counterparty?: string;   // name or organization
  orderNumber?: string;    // optional binding to order
  description: string;
  quantity?: number;       // informational only, no stock tracking
  unit?: string;          // "sheets", "packs", "kg", "pcs"
  locationType: LocationType;
  locationId?: string;
  tempLocationPhoto?: string;
  tempLocationDesc?: string;
  status: MaterialStatus;
  issuedAt?: string;
  issuedReason?: string;
  photos: string[];
  createdAt: string;
  createdBy: string;
  synced?: boolean;
}

export interface Location {
  id: string;
  name: string;           // e.g. "Rack A, shelf 3"
  qrUuid: string;         // UUID encoded in QR
  photoUrl?: string;      // reference photo of the place
  isActive: boolean;
  createdAt: string;
```

```
  synced?: boolean;
}

export interface AuditLog {
  id: string;
  userId: string;
  userName: string;
   actionType: string;        // 'create', 'update', 'delete', 'ship', 'unship', 'move', 'login'...
  entityType: string;     // 'order', 'box', 'material', 'location', 'user'...
  entityId: string;
  oldData?: any;
  newData?: any;
  comment?: string;
  createdAt: string;
}

export interface Settings {
  photoRetentionDays: number;   // 30,60,90,180,365,0 = forever
  organizationName: string;
  qrLabelSettings: {
    includeText: boolean;
    size: 'small' | 'medium' | 'large';
  };
}
```

All other types (API request/response) can be derived or placed in `types/api.ts`.

---

5. FUNCTIONAL REQUIREMENTS (DETAILED)

5.1. AUTHENTICATION & ROUTING

- Login page (`/login`): email/password. Mock validation: if email includes `admin` → role `admin`, else `operator`. Any password works.
- On success → redirect to respective layout (`/operator` or `/admin`).

- AuthContext stores user, role, login/logout functions. Provide `useAuth`.
- ProtectedRoute component: checks role and redirects to `/login` or appropriate dashboard.
- Admin can access all Operator pages (same routes). In audit log, his name is recorded, not "admin".

5.2. OPERATOR – DASHBOARD (`Dashboard.tsx`)

- Cards with counts (mock numbers):
  - Boxes in stock
  - Active orders
  - Shipments today
- Table of recent orders (last 5): order number, customer, date, boxes, status (active/completed).
- Click row → `OrderDetail`.

5.3. OPERATOR – BOX REGISTRATION (`BoxRegistration.tsx`)

5-step wizard using Ant Design Steps:

Step 1: Order Selection

- QR scanner (`QrScanner`) – reads order number from tech task.
- Manual input fallback.
- If order number not found → create new order (status `active`).
- Next enabled only if order number exists.

Step 2: Product Photo(s)

- Button "Enable Camera" – shows video feed (`useCamera`).
- Button "Take Photo" – captures base64, adds to list (thumbnails).
- At least one photo required.
- Can delete and retake.

Step 3: Sticker Photo + OCR Verification

- Take photo of handwritten sticker (order number, box number e.g. `3/8`, quantity).
- After capture, automatically run Tesseract.js on the image to extract the order number.
- Compare extracted number with the order number from Step 1.
- If mismatch → show red warning (non-blocking). Operator can:
  - Retake sticker photo

    - Ignore (optionally add comment)
    - Cancel registration
- Sticker photo saved.

Step 4: Storage Location

- Option A: Permanent shelf – scan QR code on shelf (contains UUID). Show shelf name.
- Option B: Temporary spot – take a photo of the surroundings, enter text description (required).
- Display selected location.

Step 5: Box Details & Save

- Box number (text field, e.g. `2/8`).
- Quantity (number).
- "Register" button – calls mock API, resets wizard, success message.

Batch Registration (add a button "Batch Registration" on Step1):

- Input: total boxes N.
- Single common product photo.
- Single common sticker photo (OCR check done once).
- Quantity mode:
    - Same for all – one number.
    - Individual – table input for N values.
- Location mode:
    - All same place – one location scan.
    - Distribute – groups: specify count per group + location.
- Start box number (default 1). System generates `1/N, 2/N, ..., N/N`.
- Creates N boxes at once.

5.4. OPERATOR – ORDER SEARCH & VISUAL SEARCH (`OrderSearch.tsx`)

- Input + "Search" button (by order number).
- Button "Scan QR" – opens `QrScanner`, fills input, auto-search.
- Button "Search by Photo" – opens `PhotoGallery` (grid of all product photos from registered boxes, newest first, one per order). Click photo → navigates to `OrderDetail` of that order.
- Search results as list: order number, customer, date, box count, status tag. Click → detail.

5.5. OPERATOR – ORDER DETAIL (`OrderDetail.tsx`)

- Order header info: number, customer, created, status, summary (total boxes / in stock / shipped).
- Boxes table (AntD Table) columns:
  - Box number
  - Quantity
  - Location (text + icon)
  - Product photo thumbnail (click to enlarge)
  - Status tag (in stock / shipped)
  - Actions:
    - "Ship" (single, if in stock)
    - "Move" – opens location selection (same as Step4)
    - "Edit" – for Operator only quantity; for Admin all fields
    - "Report Problem" – modal with problem type (dropdown) or free text, attach defect photo; box gets ⚠️ icon.
- Multi-select checkboxes (only in-stock boxes) + "Ship selected" button.
- If all boxes shipped → order status automatically becomes `completed`.
- "Print Act" button – generates PDF with list of shipped boxes, order number, date, operator name (stub implementation using `jspdf`).
- History tab – timeline of all events for this order and its boxes (creation, moves, shipments, edits, problems) with user name and timestamp.

5.6. OPERATOR – MATERIALS WAREHOUSE (`MaterialsPage.tsx`)

Tab 1: Receiving

- Form:
  - Counterparty (input with autocomplete – mock list; can add new on the fly).
  - Order number (optional) – to link client-supplied goods.
  - Material description (textarea, required).
  - Photos (one or more, capture via camera or upload).
  - Storage location (permanent QR or temporary spot, same as Step4).
- "Receive" button – creates material with status `in_stock`.

Tab 2: Search & Issue

- Search bar – searches in description, counterparty, order number.
- Table of materials: description, counterparty, receipt date, location, status (in_stock/issued), photo thumb.
- Actions:
  - "Issue" (only if in_stock) – reason (Client / Production / Other) + optional comment → status becomes `issued`.
  - "Move" – change location.

No partial issue / stock tracking. Whole unit is issued/returned.

5.7. ADMIN – SHELF MANAGEMENT (`AdminLocations.tsx`)

- List of all permanent locations (name, QR UUID, active/inactive, created date).
- "Create new location" modal:
  - Name (required).
  - Photo of the place (required, capture or upload).
  - On save: generate UUID, create PDF label (function `generateLocationLabel` in `utils/pdf.ts`).
  - PDF contains: large name text, QR code with UUID (or URL), date, ID.
  - PDF is downloaded automatically.
- Actions per location:
  - "Download QR" – regenerate PDF with same UUID.
  - "Deactivate" / "Activate" – inactive locations cannot be scanned (API returns error).
  - "Delete" – check if any boxes/materials are attached:
    - If yes → warning with count, confirm; after deletion those items lose location reference.
    - If no → permanent delete.

5.8. ADMIN – USER MANAGEMENT (`AdminUsers.tsx`)

- List users: name, email, role, status (active/blocked), last login (stub).
- "Invite user" modal: name, email, role → adds to list (mock).
- Edit: change role, block/unblock.
- Reset password (mock).
- Delete not recommended – block instead.

5.9. ADMIN – MANUAL ORDER MANAGEMENT (`AdminOrders.tsx`)

- List all orders with filters (number, customer, date range, status).
- "Create order" – input unique order number, optional customer → creates empty order.
- Edit order – can change order number (cascades to all boxes), customer.
- Delete order – only if no boxes attached.

5.10. ADMIN – EDIT BOX & UNSHIP

- Admin can open any box detail and click "Edit".
- All fields editable:
  - Reassign to another order.
  - Change box number, quantity.
  - Add/delete product photos.
  - Replace sticker photo.
  - Change location.
- Unship: button for boxes with status `shipped`. Confirm, optional comment → status back to `in_stock`, order becomes active again if was completed.

5.11. ADMIN – AUDIT LOG (`AdminAudit.tsx`)

- Table with all `AuditLog` entries (mock data).
- Filters: date range, user, action type, order number.
- Columns: timestamp, user, action, entity, details (old/new values), comment.
- No delete/clear.

5.12. ADMIN – SYSTEM SETTINGS (`AdminSettings.tsx`)

- Photo retention period (select: 30,60,90,180,365 days, 'never').
- QR label settings (size, include text).
- Organization name (appears in PDF acts).
- Save to localStorage / mock API.

5.13. ADMIN – STATISTICS DASHBOARD (`AdminStats.tsx`)

- KPI cards: active orders, boxes in stock, materials in stock, shipments today/week/month.
- Simple chart (Recharts or Chart.js stub) – daily receipts & shipments.
- Table of long-staying orders (e.g. >7 days).
- Operator activity table: operator name, # packings, # shipments, # moves (mock).

---

6. CORE COMPONENTS & UTILS (MUST IMPLEMENT)

6.1. `useCamera` hook (`hooks/useCamera.ts`)

```typescript
export function useCamera(facingMode: 'user' | 'environment' = 'environment') {
  const videoRef = useRef<HTMLVideoElement>(null);
  const [isActive, setIsActive] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const start = async () => { ... };
  const stop = () => { ... };
  const capture = (): string | null => { ... }; // returns base64 JPEG

  return { videoRef, start, stop, capture, isActive, error };
}
```

6.2. `QrScanner` component (`components/QrScanner.tsx`)

Wrapper over `Html5Qrcode`. Props:

- `onScan: (text: string) => void`
- `onError?: (error: string) => void`
- `fps?: number` (default 10)
- `qrbox?: number` (default 250)

Auto-starts on mount, auto-stops on unmount.

6.3. `PhotoGallery` component (`components/PhotoGallery.tsx`)

- Grid of product photos (from all boxes, grouped by order, one representative photo per order).
- Each tile: photo thumbnail + order number.
- Click → navigates to `OrderDetail` of that order.
- Used in `OrderSearch` for visual search.

6.4. OCR utility (`utils/ocr.ts`)

```typescript
export async function recognizeOrderNumber(imageBase64: string): Promise<string | null> {
  // Initialize Tesseract worker (eng+rus), run on image, extract digits.
```

```
  // Return cleaned number string or null.
}
```

6.5. PDF utilities (`utils/pdf.ts`)

- `generateLocationLabel(name: string, uuid: string): Promise<Blob>` – uses `jspdf` + `qrcode`.
- `generateShipmentAct(orderNumber: string, boxes: Array<{ number: string; quantity: number }>, operatorName: string): Promise<Blob>` – uses `jspdf-autotable`.

---

7. OFFLINE STORAGE (DEXIE) – `store/db.ts`

```typescript
import Dexie from 'dexie';

export class WarehouseDB extends Dexie {
  orders: Dexie.Table<Order, string>;
  boxes: Dexie.Table<Box, string>;
  materials: Dexie.Table<Material, string>;
  locations: Dexie.Table<Location, string>;
  syncQueue: Dexie.Table<{ id: string; url: string; method: string; body: any; timestamp: number }, string>;

  constructor() {
    super('WarehouseDB');
    this.version(1).stores({
      orders: 'id, number, status, createdAt, synced',
      boxes: 'id, orderId, status, locationId, synced',
      materials: 'id, status, locationId, synced',
      locations: 'id, qrUuid, isActive, synced',
      syncQueue: 'id, timestamp',
    });
  }
}
```

```
export const db = new WarehouseDB();
```

Sync is NOT required for MVP, but the table structure must be present. All CRUD operations in mock API should also update Dexie if online status is simulated (we can keep it simple – just use in-memory mock).

---

8. MOCK API SERVICES (`services/`)

Implement each service as a set of functions (not actual HTTP) that return Promises with mock data. Simulate network delay with `setTimeout` (300–500ms). Store data in global variables or `localStorage` to persist across reloads.

Minimal required mock data (pre-seed on first load):

- Users: admin@test.com (Admin), operator@test.com (Operator).
- Locations: 3 permanent shelves.
- Orders: #12345 (client "Ivanov", 5 boxes: 3 in stock, 2 shipped), #12346 (client "Romashka", 2 boxes in stock).
- Materials: "Gray paper 70x100, 10 sheets" (counterparty "Igor"), "Die-cut form" (counterparty "Ivanov").

---

9. PWA & MANIFEST

Configure `vite.config.ts` with `VitePWA` plugin:

```typescript
VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'Printing House Warehouse',
    short_name: 'Warehouse',
    description: 'Manage finished goods and materials',
    theme_color: '#1890ff',
    background_color: '#ffffff',
    display: 'standalone',
    icons: [ /* include a few PNG icons from public/ */ ]
  },
  workbox: {
```

```
    globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}']
  }
})
```

Create a simple `public/icon-192.png` and `public/icon-512.png` (you can use base64 placeholder or generate from `vite.svg`).

---

10. GLOBAL STYLES & UX REQUIREMENTS

- `global.scss`:
  - All buttons, inputs, tabs: `large` size.
  - Media queries for mobile: reduce padding, full-width video.
  - Camera preview: `width: 100%; max-width: 400px; background: black; border-radius: 8px;`.
  - High contrast, large touch targets.
- Use Ant Design's built-in RTL? No, LTR.
- All async operations must show loading indicators (`Spin`, `Button loading`).
- Form validation: required fields highlighted, tooltips.
- Use `message.success`, `message.error`, `notification` for feedback.

---

11. WHAT THE GENERATED PROJECT MUST INCLUDE

✅ A fully working React + TypeScript + Vite + Ant Design PWA, installable.
✅ All pages and components as described, with stubbed business logic but functional UI.
✅ Camera access, QR scanning, OCR (basic) implemented.
✅ PDF generation for shelf labels and shipment acts (basic, but functional).
✅ Mock API with seeded data.
✅ Offline IndexedDB schema ready (no sync logic required).
✅ Clean code, strong typing, no `any` (except unavoidable).
✅ Responsive and mobile-friendly.

DO NOT generate real backend, real authentication, real cloud storage. Everything stays in the browser (memory or `localStorage`).
DO NOT add extra libraries beyond the stack.

---

12. FINAL INSTRUCTIONS

Generate the entire project in one go. Follow the folder structure and file names exactly. Every file must contain valid, runnable code. Use named exports. Include minimal comments where necessary (English). Make sure all imports are correct.

Start with `package.json` and then generate all files in the order they are needed. The project must be runnable after `npm install && npm run dev`.

This is the complete specification. No more details will be provided. Execute the prompt now.